HW 4.3 - Reproducible Data Analysis

Connor Box

2026-04-14

Busiest Airports Analysis

Data Overview

The data used in this analysis comes from Wikipedia's list of the busiest airports by passenger traffic. It includes yearly passenger totals for six major airports—Atlanta (ATL), Frankfurt (FRA), Beijing Daxing (PKX), Dubai (DXB), Tokyo Haneda (HND), and London Heathrow (LHR)—from 2020 through 2025. After cleaning, the dataset was organized into a tidy format so that each row represents a single airport in a given year. This makes it easier to compare changes over time and across locations.

Table Results

Table 1 shows passenger totals for each airport across the selected years. Looking at the data in table form makes it easier to compare exact values and see which airports consistently handle the most traffic.

Table 1: Passenger Traffic for Selected Major Airports, 2020–2023

Year	Airport	IATA Code	Passengers
2020	Hartsfield–Jackson Atlanta International Airport	ATL	42918685
2021	Hartsfield–Jackson Atlanta International Airport	ATL	75704760
2022	Hartsfield–Jackson Atlanta International Airport	ATL	93699630
2023	Hartsfield–Jackson Atlanta International Airport	ATL	104653451
2020	Dubai International Airport	DXB	25836771
2021	Dubai International Airport	DXB	29110609
2022	Dubai International Airport	DXB	66069981
2023	Dubai International Airport	DXB	86994365
2020	Frankfurt Airport	FRA	18768601

Year	Airport	IATA Code	Passengers
2021	Frankfurt Airport	FRA	24812849
2022	Frankfurt Airport	FRA	48918482
2023	Frankfurt Airport	FRA	59355389
2020	Tokyo Haneda Airport	HND	31055210
2021	Tokyo Haneda Airport	HND	25876429
2022	Tokyo Haneda Airport	HND	50290705
2023	Tokyo Haneda Airport	HND	78719302
2020	Heathrow Airport	LHR	22111469
2022	Heathrow Airport	LHR	61614508
2023	Heathrow Airport	LHR	79183364
2021	Beijing Daxing International Airport	PKX	25051012

Visualization and Interpretation

Figure 1 shows how passenger traffic changes over time for the six airports. Compared to the table, the graph makes overall trends easier to see.

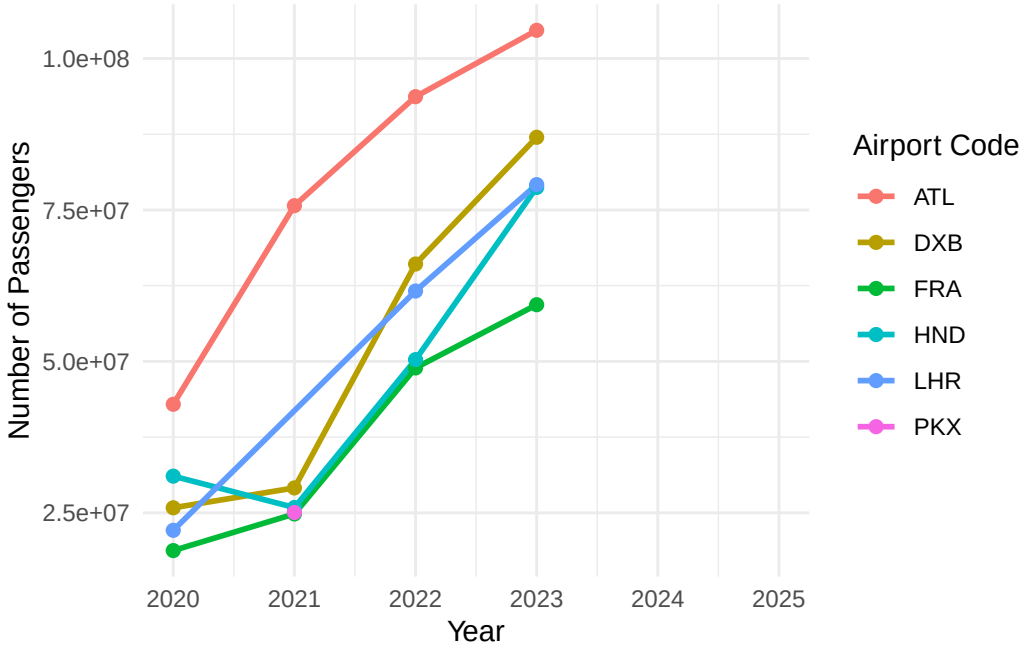

Figure 1: Passenger Trends for Six Major Airports, 2020–2023

From Table 1 and Figure 1, it is clear that passenger traffic varies quite a bit between airports

and across years. Airports like Atlanta and Dubai tend to have higher totals, while others are lower overall.

One noticeable pattern is the drop around 2020, followed by a steady recovery. This likely reflects the impact of global travel disruptions and how traffic gradually returned in the following years. The visualization makes this recovery pattern much easier to see compared to just looking at raw numbers.

Monte Carlo Numerical Integration

Method Overview

Monte Carlo integration is a method used to estimate the value of an integral using random sampling. In this case, the goal is to estimate the area under the Beta(2,2) curve from 0 to 1. Random points are generated within a rectangle that contains the curve, and the proportion of points that fall under the curve is used to estimate the area.

Simulation Results

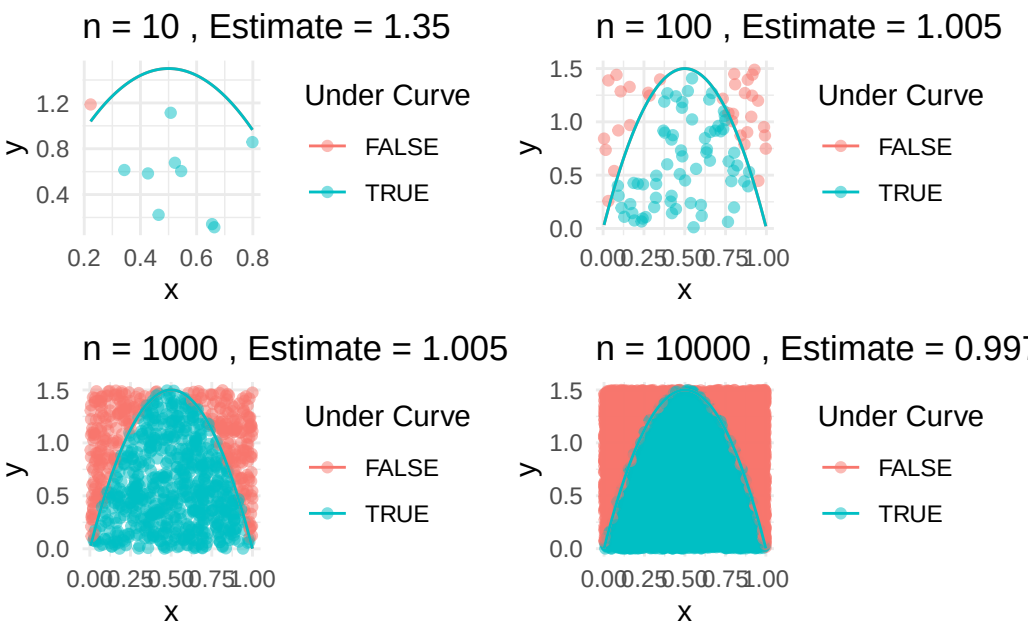

Figure 2: Monte Carlo Estimates for Increasing Sample Sizes

Interpretation

Figure 2 shows how the estimate improves as more points are used. With only 10 points, the estimate is pretty rough and not very reliable. As the number increases to 100 and 1000, the estimate becomes more stable and closer to the actual value.

By 10,000 points, the estimate is much more consistent. This shows how increasing the sample size improves accuracy, which is a key idea behind Monte Carlo methods.

The exact value of the integral of the Beta(2,2) distribution over the interval from 0 to 1 is 1. This is supported by the simulation results, as the estimates approach 1 as the number of sampled points increases.

Planning and Prompting GenAI Tools

Data Wrangling Plan

The calcium dataset contains measurements from two groups of women over multiple years, but it is not in a tidy format. The first set of columns corresponds to the null treatment group, while the second set corresponds to the diet and exercise group. Each group contains measurements taken at several time points.

My plan is to first separate the two groups and then restructure the data from a wide format into a long format. This will involve combining the measurement columns into a single variable and creating new variables to represent both the treatment group and the year of measurement. Once the data is in a tidy format, I will be able to more easily compare trends over time between the two groups.

After tidying the data, I plan to create a visualization that shows how calcium levels change across years for each group. A line plot will be used, with time on the x-axis and calcium measurement on the y-axis, and different colors used to distinguish between the treatment groups.

GenAI Output (Plan-Based Prompt)

Prompt Used:

I created the following plan for cleaning and analyzing a dataset:

- Separate the two groups (null treatment and diet/exercise)
- Reshape the data from wide format to long format
- Create variables for treatment group and year
- Combine all calcium measurements into a single column
- Create a plot comparing calcium levels over time between the two groups

Apply this plan to the uploaded dataset and describe how each step would be carried out.

Response:

The dataset contains calcium measurements for two groups across multiple years in a wide format. The first step is to separate the data into the null treatment group and the diet/exercise group.

Next, the data should be reshaped into a long format by combining the measurement columns into a single column and creating a variable to represent the year of measurement. Another variable should be added to indicate the treatment group.

Once the data is in tidy format, it can be visualized using a line plot. The x-axis should represent time, and the y-axis should represent calcium measurements. Different colors can be

used to distinguish between the two treatment groups, allowing for comparison of trends over time.

GenAI Output (Generic Prompt)

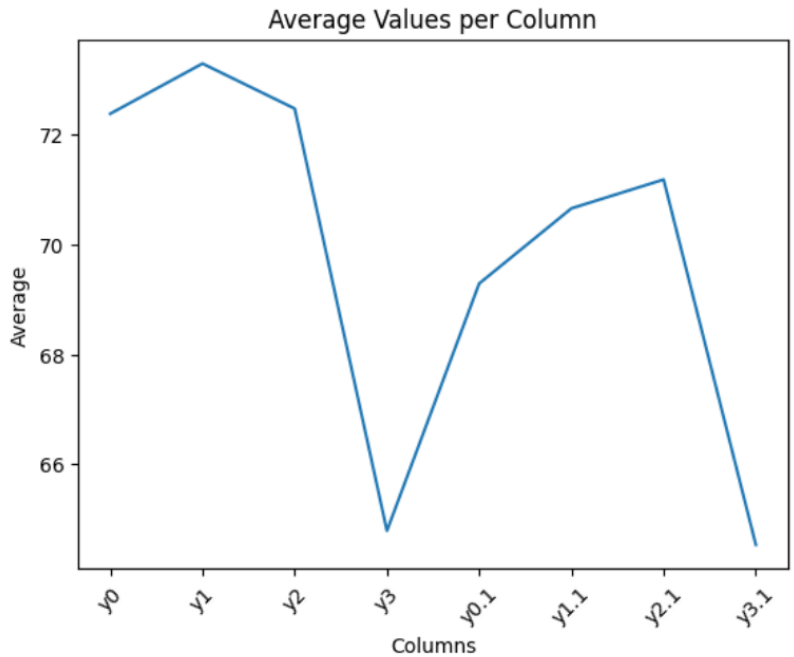

Figure 3: Calcium Levels Over Time

The generic prompt produced a visualization that shows the average value of each column in the dataset. Each point represents the mean calcium measurement for a specific column (such as y0, y1, etc.), allowing for a general comparison of overall values across the dataset.

The response explained that this plot highlights differences in average calcium levels, with most values clustering around similar ranges and a few lower values appearing in certain columns. While this provides a basic overview, it does not fully capture how calcium levels change over time or differ between the treatment groups.

Comparison of Results

The plan-based response was more useful because it directly addressed the structure of the calcium dataset. It identified the need to separate the two treatment groups and reshape the data into a long format, which is necessary for analyzing changes over time.

In contrast, the generic response focused only on calculating average values for each column. While this provides a simple summary, it does not reflect the longitudinal nature of the data or allow for meaningful comparison between treatment groups.

Compared to both GenAI outputs, my own approach better reflects the goals of the assignment by focusing on trends over time and differences between groups. This highlights that having a clear plan leads to more relevant results, while generic prompts tend to produce more limited insights.

Reflection

This assignment helped me understand how to build a full reproducible analysis from start to finish. One of the biggest things I learned was how to separate code from the main report while still producing tables and graphs.

Working with the airport data also showed how important data cleaning is before doing any analysis. The Monte Carlo section helped reinforce how simulation can be used to approximate values when an exact solution is difficult.

I also learned how much prompt quality matters when using AI tools. Being more specific leads to better and more useful results.

Appendix A: GenAI Usage

Usage 1: Plan-Based Prompt

Tool Used: ChatGPT (GPT-5.3)
Date: April 2026

Prompt Used:
I created the following plan for cleaning and analyzing a dataset: - Separate the two groups (null treatment and diet/exercise) - Reshape the data from wide format to long format - Create variables for treatment group and year - Combine all calcium measurements into a single column - Create a plot comparing calcium levels over time between the two groups

Apply this plan to the uploaded dataset and describe how each step would be carried out.

Response:
The dataset contains calcium measurements for two groups across multiple years in a wide format. The first step is to separate the data into the null treatment group and the diet/exercise group.

Next, the data should be reshaped into a long format by combining the measurement columns into a single column and creating a variable to represent the year of measurement. Another variable should be added to indicate the treatment group.

Once the data is in tidy format, it can be visualized using a line plot. The x-axis should represent time, and the y-axis should represent calcium measurements. Different colors can be used to distinguish between the two treatment groups, allowing for comparison of trends over time.

Usage 2: Generic Prompt

Tool Used: ChatGPT (GPT-5.3)
Date: April 2026

Prompt Used:
Create a visualization of the calcium dataset and describe the results.

Response:
The visualization shows the average value of each column in the dataset. Each point represents the mean calcium measurement for a specific column (such as y0, y1, etc.), allowing for a general comparison of overall values across the dataset.

The plot highlights differences in average calcium levels, with most values clustering around similar ranges and a few lower values appearing in certain columns. This provides a general overview of the dataset, but does not fully capture how calcium levels change over time or differ between treatment groups.

Appendix B: Code Appendix

```r
# Load packages
library(readr)
library(dplyr)
library(tidyr)
library(ggplot2)
library(knitr)
library(patchwork)

# Read in airport data
airports_tidy <- read_csv("data/airports_tidy.csv")

# Create airport table for report
airports_table <- airports_tidy %>%
  filter(!is.na(passengers)) %>%
  mutate(passengers = round(passengers, 0)) %>%
  arrange(iata, year) %>%
  select(year, airport, iata, passengers)

# Create airport trend plot
airport_plot <- ggplot(
  airports_tidy,
  aes(x = year, y = passengers, color = iata)
) +
  geom_line() +
  geom_point()

# Function to generate Monte Carlo points
monte_carlo_points <- function(n, xmin, xmax, ymin, ymax) {
  tibble(
    x = runif(n, xmin, xmax),
    y = runif(n, ymin, ymax)
  )
}

# Generate Monte Carlo datasets
points_10 <- monte_carlo_points(10, 0, 1, 0, 1.5)
points_100 <- monte_carlo_points(100, 0, 1, 0, 1.5)
points_1000 <- monte_carlo_points(1000, 0, 1, 0, 1.5)
points_10000 <- monte_carlo_points(10000, 0, 1, 0, 1.5)
```

```r
# Read in calcium data
calcium <- read_csv("data/calcium.csv")

# Reshape calcium data from wide to long format
calcium_long <- calcium %>%
  pivot_longer(
    cols = everything(),
    names_to = "variable",
    values_to = "calcium"
  )

# Create generic calcium plot
calcium_plot <- ggplot(calcium_long, aes(x = variable, y = calcium)) +
  stat_summary(fun = mean, geom = "point") +
  labs(
    title = "Calcium Levels Over Time",
    x = "Measurement",
    y = "Calcium"
  )
```